Computers & Security 159 (2025) 104670

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

VDExplainer: Sequential decision-making and probability sampling guided statement-level explanation for vulnerability detection☆

Weining Zheng, Xiaohong Su*, Yuan Jiang, Hongwei Wei, Wenxin Tao

Harbin Institute of Technology, Harbin, China

ARTICLE INFO

Keywords:
Vulnerability detection
Software security
Pre-trained model
Deep learning
Explainable AI
Sequential decision-making
Probability sampling

ABSTRACT

Most existing deep learning (DL) based vulnerability detection methods, including pre-trained models, are coarse-grained binary classification methods that lack the interpretability for detection results. Although the explanation of deep learning has received significant attention, there is little research on the explanation of pre-trained model-based vulnerability detection methods. Therefore, we focus on providing statement-level interpretability for these vulnerability detection models to help developers understand the vulnerabilities. More specifically, given a vulnerable code detected by the model, our task is to find the set of vulnerability-related statements that lead to the prediction. Inspired by the manual code review process, this paper proposes a framework for explaining vulnerability detection called VDExplainer. VDExplainer includes an explorer that uses sequential decision-making and probability sampling to find the combination of vulnerability-related statements and a navigator that helps reduce the search space by learning the vulnerability patterns. It is worth noting that the navigator is trained in advance and then integrated with the explorer, further enhancing the efficiency and effectiveness of VDExplainer. Extensive experiments on the semi-synthetic dataset and the widely used real-world project dataset show that VDExplainer achieves superior performance, outperforming current state-of-the-art methods.

1. Introduction

Deep-learning-based vulnerability detection techniques have made significant progress and have been widely explored on the sequence-based (Li et al., 2018, 2021c,b), tree-based (Zhang et al., 2019; Yang et al., 2021) and graph-based (Zheng et al., 2021a; Li et al., 2021a) code representations. In recent years, pre-trained models have shown excellent performance in code-related tasks, and as a result, some researchers have begun to apply these models to code vulnerability detection tasks as well (Hanif and Maffeis, 2022). However, most current deep-learning-based vulnerability detection methods, including pre-trained models, remain essentially coarse-grained binary classification methods. These methods can provide the probability that a file, function, or even a code segment (also called code gadget (Li et al., 2018) or SeVC ((Li et al., 2021c) in the original paper) contains vulnerabilities.

Despite the good results achieved, all these models are black boxes. How the models understand the semantics of vulnerabilities in the code and the key vulnerability-related statements that influence their predictions are not visible to developers. It is therefore important to explain these DL-based vulnerability detection models to understand which statements may potentially lead to the vulnerabilities. Moreover, it is difficult for developers to understand vulnerabilities manually relying on expert knowledge, which may result in vulnerabilities not being patched promptly.

There are already some approaches that provide explanations of DL-based vulnerability detection models. For example, Zou et al. proposed a model explanation method based on heuristic search, which uses heuristic search to obtain combinations of tokens that have a significant impact on model predictions by masking the tokens in the code and comparing the changes in model predictions before and after the masking (Zou et al., 2021). Another vulnerability detection method with fine-grained interpretations, i.e. IVDetect (Li et al., 2021a) uses GNNexplainer (Ying et al., 2019) to mask edges in the graph-based representation of their samples. By comparing the difference between detection results before and after masking, the importance of each edge is obtained, and important edges are composited into a vulnerability-related PDG (Program Dependence Graph) subgraph, which in turn explains the detection results. Although the above methods can provide explanations for DL-based vulnerability detection models, they still have some weaknesses. For example, PGExplainer (Luo et al., 2020)

☆ This document is the results of the research project funded by the National Science Foundation.
* Corresponding author.
E-mail address: sxh@hit.edu.cn (X. Su).

https://doi.org/10.1016/j.cose.2025.104670
Received 5 April 2025; Received in revised form 4 August 2025; Accepted 13 September 2025
Available online 25 September 2025

highlights critical limitations of GNNExplainer: it focuses on local interpretability by generating customized explanations for individual instances independently, making it hard to generalize to other nodes; it requires retraining for each explanation, leading to high time overhead in practical scenarios; and its explanatory motifs are not learned end-to-end with a global view of the entire GNN model.

Therefore, we believe such approaches have two main weaknesses: (1) They incur high explanation costs (especially time overhead) due to the need to search for important code elements through detection result feedback. (2) They focus on explaining isolated individual samples, making it difficult to learn generalized features from multiple samples, which requires per-explanation retraining when generalizing to other instances.

To mitigate the above problems, we propose VDExplainer, a model-independent, statement-level explainer for interpreting vulnerability detection models based on DL and pre-trained model. For the vulnerable code predicted by the model, the objective of VDExplainer is to extract a collection of vulnerability-related statements that tend to be interdependent and lead the model to predict the code as vulnerable. For vulnerability analysis, we believe that extracting statements related to vulnerabilities is more useful in helping developers understand the causes of vulnerabilities, as seen in Section 2.1. Specifically, VDExplainer combines an explorer based on sequential decision and probability sampling, and a learning-based navigator, leveraging their strengths to accelerate explanation efficiency while improving accuracy. Among them, the learning-based navigator will summarize all vulnerable samples during the training phase to learn the semantic information about the vulnerabilities they contain. Once the training is completed, it can provide reference probabilities for the explorer to dynamically update the set of candidate vulnerability-related statements, thereby extracting vulnerability patterns and accelerating the interpretation speed. The explorer based on sequential decision-making and probability sampling, on the other hand, can simulate the manual code review process by judging and searching for statements in sequence with the help of the navigator, thus providing the best combination of vulnerability-related statements for the vulnerable samples.

To validate VDExplainer, we conducted extensive experiments on three datasets: (1) Big-Vul, a real-world project dataset with fine-grained statement-level labels; (2) a semi-synthetic dataset combining real-world and synthetic code snippets; and (3) D2 A, a dataset with recorded defect paths. We compared VDExplainer against 8 state-of-the-art explanation methods (e.g., Attention, SHAP, GNNExplainer) on 5 vulnerability detection models, including pre-trained models (LineVul, VulBERTa-CNN) and DL-based models (SySeVR, Devign, IVDetect). Key results show that: First, VDExplainer outperforms all baselines in key metrics (e.g., 5-accuracy, line coverage) across datasets. For example, on Big-Vul, it achieves 75% 5-accuracy for LineVul, 25%–41% higher than model-dependent methods (Attention, LIG) and 36%–55% higher than model-independent methods (SHAP, Random). Second, Its advantages stem from the navigator's dynamic context integration and probability sampling, as verified by ablation studies (e.g., 30% noise reduces 5-accuracy by 22%). Finally, VDExplainer maintains superiority even on defect-path datasets (D2 A), with 69% 5-accuracy for LineVul, confirming its generalizability. These results demonstrate that VDExplainer sets a new state-of-the-art for statement-level vulnerability explanation, with significant implications for improving the interpretability of deep learning-based vulnerability detection.

The main contributions of our work include:

- We propose VDExplainer, a new statement-level explainer for pre-trained model-based and DL-based vulnerability detection models. Inspired by the manual code review process, VDExplainer models the statement-level vulnerability explanation task as a sequential decision problem of vulnerability-related statements, rather than a classification problem. It combines a probability sampling and sequential decision-based explorer with a learning-based navigator to improve efficiency while providing more accurate explanations.

```
1 const EVP_CIPHER *EVP_aes_192_cfb8()
2 {
3  int stonesoup_oc_i = 0;
4  int stonesoup_file_desc;
5  char stonesoup_buffer[128];
6  char stonesoup_input_buf[128] = {0};
... ……
13 if (__sync_bool_compare_and_swap(&disjoined_tamassee,0,1)) {
14    if (mkdir("/opt/stonesoup/workspace/lockDir",509U) == 0) {
...       ……
18       if (agonia_terrazzos != 0) {
...          ……
25          memset(stonesoup_buffer,'x',128);
26          stonesoup_buffer[127] = 0;
27          stonesoup_file_desc = open(sallyman_unannoyingly,0);
28          if (stonesoup_file_desc > -1) {
29             tracepoint(stonesoup_trace, trace_point, "CROSSOVER-POINT:BEFORE");
30             read(stonesoup_file_desc,stonesoup_input_buf,128);
31             close(stonesoup_file_desc);
32             tracepoint(stonesoup_trace, trace_point, "CROSSOVER-POINT:AFTER");
33             tracepoint(stonesoup_trace, trace_point, "TRIGGER-POINT:BEFORE");
34             strcpy(stonesoup_buffer,stonesoup_input_buf);
...             ……
46       }
47    }
48 }
49 return OPENSSL_ia32cap_P[1]&(1 << 57 - 32)?&aesni_192_cfb8:&aes_192_cfb8;
50}
```

Vulnerable statement Vulnerability-related context

Fig. 1. Motivation example. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

- We design a general explanation framework VDExplainer which can be applied flexibly and efficiently to most vulnerability detection methods based on DL and pre-trained models. VDExplainer consists of an explorer and a learnable navigator. The navigator can be trained with or without fine-grained labels.
- We conduct an extensive evaluation of VDExplainer on the semi-synthetic dataset and the real-world dataset to verify the effectiveness and efficiency of the proposed method. Experimental results show that VDExplainer outperforms other state-of-the-art methods in explanation performance. To help researchers in this field and to support the open science community, we have made the source code of VDExplainer and the datasets used available to the public at https://figshare.com/s/9f6a9f553a9dc30029c9.

2. Motivation

2.1. Motivational example

Fig. 1 shows the vulnerable code in the National Vulnerability Database (NVD) with the OpenSSL project as the base program, number 153804. "This code implements a file read of 128 characters which does not properly null terminate the copied string if the original string is 128 characters in length or greater". The red box shows the vulnerable statement and the pink boxes show the vulnerability-related context, both of which are vulnerability-related statements. We refer to the set of statements related to causing the vulnerability as the vulnerability pattern in code. These statements involve two variables, *stonesoup_buffer* and *stonesoup_input_buf*, defined at lines 5 and 6. Next, variable *stonesoup_buffer* is initialized at lines 25 and 26, while a file is opened at line 30 and the first 128 characters are read to an internal buffer of size 128 allocated on the stack. Finally, the code copies this buffer into another buffer of size 128 allocated by the stack via *strcpy* at line 34. At this point, however, if the original file read did not copy a null terminator, the *strcpy* will copy everything it finds in memory after the input string until it encounters a null terminator. This will cause a buffer overflow vulnerability. Based on the vulnerable code in Fig. 1, we can make the following two observations: (1) For vulnerable code, providing vulnerability-related statements can effectively reduce the difficulty for developers to locate and fix vulnerabilities. (2) Vulnerability patterns are not isolated vulnerable statements, but a combination of vulnerable statements and their context. For example, although the location of the vulnerability in the code above is line 34, the code reviewer needs to combine all the statements in the boxes related to the vulnerability when analyzing the cause of the vulnerability.

Consequently, we can draw the following two observations from the above process: (1) Providing detailed explanations at the level of the vulnerability-related statement can effectively reduce the difficulty for developers to locate and fix vulnerabilities. (2) If we can simulate the above process of manual code review to discover vulnerabilities by combining with vulnerability context, we can obtain more effective and accurate explanations of vulnerabilities. Therefore, from the above observations, we summarize two design ideas for explainers: (1) For vulnerable codes, an explainer that provides fine-grained explanation at the statement level can effectively reduce the difficulty that developers face in discovering and fixing vulnerabilities. (2) If an explainer can simulate the above vulnerability analysis process, by combining with vulnerability context, it can achieve more effective and accurate explanations of vulnerabilities. Therefore, we propose VDExplainer, a model-independent, statement-level explainer for DL-based and Pre-trained model-based vulnerability detection models.

2.2. Motivation for explorer

The distribution of human attention during reading is more inclined to read sequentially at first, then skip unimportant content and focus only on the most important content (Zheng et al., 2019). Taking vulnerability code review as an example: security experts examine the code and locate the location of vulnerability-related statements, usually after analyzing the entire code to exclude some obviously vulnerability-unrelated statements. Then, they focus on examining key statements and their context suspected to contain vulnerabilities. Finally, based on these statements, more specific information about vulnerabilities is collected for subsequent decision-making. Still taking the vulnerable code shown in Fig. 1 as an example, in order to understand the vulnerabilities in the code, security experts only need to focus on the statements related to variables *stonesoup_buffer* and *stonesoup_input_buf*, check them from the beginning, and gradually collect information until the location where the vulnerability occurs. Inspired by the process of manual analysis and vulnerability discovery, we build an explorer based on sequential decision-making and probability sampling to transform vulnerability detection from a classification problem to a sequential decision problem. This explorer simulates the process of manually analyzing vulnerability code and better integrates the context of the vulnerability to provide more accurate explanations.

2.3. Motivation for navigator

A popular class of methods for interpreting DL-based vulnerability detection models are model-dependent methods. These methods search for important features implied in input samples by analyzing the parameters of the model itself, which can be the model gradient or the attention score (if the method uses an attention mechanism). These methods are easy to implement, have high computational efficiency and low running time. However, according to the experimental results of currently available studies, their method still has room for improvement in accuracy. (For example, in the experimental results of LineVul, the top 5 accuracy of several methods did not exceed 50% (Fu and Tantithamthavorn, 2022)).

Another class of interpretable methods is based on searching, i.e. finding combinations of code elements in a vulnerability sample and masking them (these elements can be code tokens, statements, or nodes, edges, etc. in a code graph representation). These approaches compare the detection results of the model before and after masking, and then find the combinations that have a greater impact on the detection results. Since key features are obtained by direct feedback of predicted probabilities, these methods are widely applicable and highly explainable. However, the cost of explanation is high due to the need to search a large sample feature space and perform multiple vulnerability detection and feedback. On the other hand, these methods target independent samples, making it difficult to learn generalized features from multiple samples, increasing the overhead. Finally, in order to batch input during the detection process, DL-based and Pre-trained model-based vulnerability detection models need to truncate samples. For example, the LineVul (Fu and Tantithamthavorn, 2022) and VulBERTa-CNN (Hanif and Maffeis, 2022) methods set the maximum number of tokens for sample input to 512 and discard tokens that exceed this limit, i.e. the above methods are unable to handle samples that exceed the truncation limit.

Therefore, we build learning-based navigator to support VDExplainer. First, the navigator can speed up the search efficiency of explorer and thus reduce the workload by providing navigation probabilities to the explorer during sampling. Second, the learning-based navigator can be pre-trained on the training set and learn vulnerability patterns in combination with the semantic information extracted by the vulnerability detection model to help the explorer provide more accurate explanations.

3. Methodology

In this section, we introduce how to identify a meaningful set of vulnerability-related statements from the detected vulnerability code. Explanation methods should continuously search for combinations of statements in the search space composed of code statements in order to obtain the best combination for explanation. For this reason, we propose VDExplainer, which is an explanation framework composed of explorer and navigator to obtain explanations of vulnerabilities effectively and efficiently, as shown in Fig. 2. The navigator learns vulnerability patterns by training on the training set, using the semantic information provided by the vulnerability detection model. The explorer is guided by the navigator to find more specific combinations of vulnerability-related statements to explain vulnerabilities using sequential decision-making and probability sampling. Vulnerability detection models, on the other hand, are models that need to be explained, e.g., LineVul, Devign, etc. The navigator in our proposed explanation model needs to borrow the trained code representation module from the detection model to extract vulnerability features to help generate decision vectors.

3.1. Explorer

The explorer uses sequential decision-making and probability sampling methods. Sequential decision-making methods allow the explorer to simulate the manual code review process, incorporating vulnerability context to extract combinations of vulnerability-related statements. Probability sampling means that the explorer samples candidate vulnerability-related statements according to the reference probabilities provided by the navigator. Adding a certain degree of sampling randomness to the exploration process can alleviate the problem of the tendency of VDExplainer to fall into local optimum during training.

The specific exploration process of VDExplainer is as follows. First, we traverse the statements in the code line by line and make a decision on each statement in turn based on the order of the statements. The explorer performs sampling based on the vulnerable probability of each statement returned by the navigator. In order to consider the overall vulnerability pattern, the explorer uses the code representation network in the vulnerability detection model to extract semantic information from the current statement and the identified candidate vulnerability-related statements. Next, the navigator calculates the probability of vulnerability based on the decision vector derived from the semantic information. The explorer then performs probability sampling based on the vulnerability probabilities. It is important to note that the probability sampling we use is random sampling based on the discrete probability distribution. For example, assuming that the navigator provides a probability of [0,3,0,7] based on the decision vector, probability sampling would have a 30% chance of sampling 0, which means not selected, and a 70% chance of sampling 1, which means

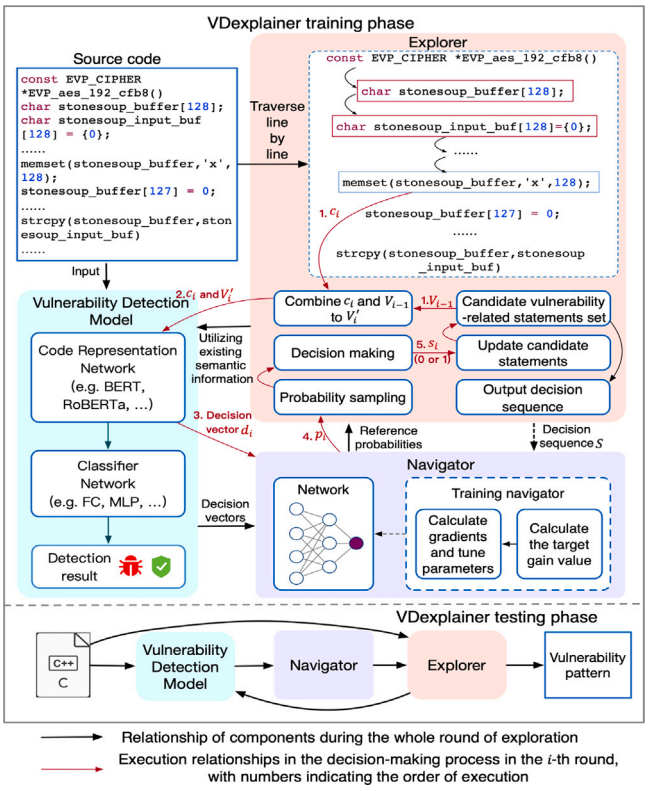

Fig. 2. The processing framework of VDExplainer. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

selected. If a statement is selected to be added to the set of candidate vulnerability-related statements, the vulnerability probability at that time is recorded. Once the number of statements in the set of candidate vulnerability-related statements exceeds the threshold, the statement with the lowest vulnerability probability will be removed, and the combination of candidate vulnerability-related statements can be obtained after completing the exploration. In the VDExplainer training process, we calculate the target gain value and its expectation gradient based on the candidate vulnerability-related statement combinations and update the parameters of the network used as the navigator by backward propagation.

Formally, Assuming that the vulnerable code consists of L lines of code statements, denoted by $C = \{c_1, c_2, \ldots, c_L\}$. The explorer traverses the statements in the code sequentially to make decisions. The decision-making process in step i is indicated by the red arrows in Fig. 2. **Step1:** the explorer combines the current statement c_i with the set of candidate vulnerability-related statements V_{i-1} that has been decided upon in the previous steps to form a candidate vulnerability pattern, denoted by V_i'. **Step2:** the code representation network generates the decision vector d_i based on the vulnerability pattern V_i' and the statement c_i. **Step3:** The navigator returns the probability p_i that the current statement is related to the vulnerability, based on the decision vector d_i. **Step4:** the explorer's decision strategy differs between the training and testing phases. In training phase, the explorer performs probability sampling based on the probability p_i provided by the navigator. This controlled randomness helps avoid local optima and enables the navigator to learn generalized vulnerability patterns (verified in Section 5.3, where replacing probability sampling with deterministic selection led to significant performance degradation). In testing phase, probability sampling is not used. Instead, the explorer directly selects the option with the highest probability. This ensures that true vulnerability-related statements are not accidentally excluded due to randomness, as the decision relies on the optimal probability derived from the pre-trained navigator. **Step5:** The explorer makes a decision based on the sampling result (training phase) or deterministic selection (testing phase). Then, if the statement is not selected, this means that statement c_i is not related to the vulnerability and there is no need to add c_i to the candidate set, let decision value $s_i = 0$. If it is selected, it means that statement c_i is related to the vulnerability. At this point, it is determined whether the number of statements in the candidate set exceeds the threshold, and if not, c_i is added to the candidate set, let $s_i = 1$. If it exceeds the threshold, p_i is compared to the sampling probability of previous candidate vulnerability-related statements, and remove the statement with the lowest probability from the candidate set. If c_i is still in the set of candidate vulnerability-related statements, let the decision value $s_i = 1$, and the decision value corresponding to the deleted statement becomes 0.

During the testing process, the explorer traverses all statements in the test sample line by line and continuously searches to update candidate vulnerability-related statements set based on the reference probability provided by the trained navigator. After traversing all statements, the explorer outputs the vulnerability pattern of the sample. Note that since the navigator has already been trained, the explorer does not need to provide a decision sequence for the navigator.

3.2. Navigator

Inspired by the process of manual code review, we design a learning-based navigator. The navigator is a component that is pre-trained to learn vulnerability patterns in code, and provide effective references for the explorer. Specifically, the navigator is a feed-forward neural network in which the input is a decision vector and the output is the vulnerable probability of the statement. It is worth noting that the structure of the navigator seems to be similar to the classifier in the vulnerability detection model, but the two are actually different. The classifier outputs the probability that the entire function contains a vulnerability and cannot give the probability that a statement is a vulnerability. The navigator can combine the intrinsic association between the current statement and vulnerability-related statements previously inferred during the sequential decision-making process to generate the probability of the current statement being related to the vulnerability. These probabilities can be used in the decision-making process of the explorer to facilitate exploration. The input characteristics and training process of the navigator are described below.

3.2.1. Decision vector

Existing models, such as the graph neural networks (GNNs) used in IVDetect (Li et al., 2021a), Devign (Zhou et al., 2019) and VulSPG (Zheng et al., 2021a), the pre-trained model used in VulBERTa-CNN (Hanif and Maffeis, 2022) and LineVul (Fu and Tantithamthavorn, 2022), the recurrent neural networks used in Vuldeepecker (Li et al., 2018) and SySeVR (Li et al., 2021c), and so on, all of them use complex network structures to learn the semantics of vulnerability code in order to extract vulnerability patterns. Therefore, we can directly use the code representation networks in these detection models to extract semantic information without having to design other networks. In addition, reusing the existing code representation networks to obtain vector representations of the codes (i.e., decision vectors) also allows the semantic information used by VDExplainer to be consistent with that used by the vulnerability detection models. Since the decision vectors are already generated by reusing the code representation networks, a simple feed-forward network can be used to generate the reference probability that the current statement is related to the vulnerability based on the decision vector.

The decision vector consists of two parts, the current statement vector and the candidate vulnerability pattern vector, which contain semantic information about the current statement and the context associated with the statement, respectively. The extraction of these vectors

requires the cooperation of the vulnerability detection model. Specifically, we feed each statement individually into the code representation network in the vulnerability detection model, using the output of the last hidden layer as a vector representation of the statement. Next, we feed the current statement into the code representation network along with the previously candidate vulnerability-related statements, again using the output of the last hidden layer as a vector of the candidate vulnerability pattern. In addition, for the graph-based representation, we first find the nodes corresponding to all statements. Then, the current statement node and the candidate vulnerability-related statement nodes are combined and connected by edges in the graph-based representation thereby forming a subgraph. Finally, the feature matrices of the statement nodes in the subgraph and the edge indexes that concern only these nodes are fed to the model to obtain the candidate vulnerability pattern vector.

Formally, the current statement c_i is fed into the code representation network, and the vector representation of the corresponding statement x_i can be derived. Next, the VDExplainer can assemble the sampled vulnerability-related statements into a list in the order of the decision process and add the current statement to the end of the list to obtain the candidate vulnerability pattern V_i'. The candidate vulnerability pattern is then sent into the code representation network to produce its corresponding vector representation h_i. Therefore, in the decision-making process at step i, the decision vector d_i is concatenated from the current statement vector x_i and the vector representation h_i of the candidate vulnerability pattern V_i', i.e. $d_i = x_i \oplus h_i$.

3.2.2. Training process

The objective of the navigator is to give the vulnerable probability of the target statement based on the decision vector, which helps the explorer to sample the statements related to the vulnerability to speed up the search. The purpose of the training is to enable the navigator to gain a better understanding of the general pattern of vulnerability.

Given a decision vector d_i, the navigator can predict the vulnerable probability p_i based on d_i, then the explorer samples the statement c_i in the sequence of statements in the $\{0,1\}$ space based on p_i, and the result of the sampling is denoted by s_i, where s_i is 1 to indicate that s_i is related to the vulnerability, and 0 to indicate that statement c_i is unrelated to the vulnerability. The results of all the L lines of statements sampled in the code form a decision sequence $S = \{s_1, \ldots, s_i, \ldots, s_L\}$, using this decision sequence as the result of statement-level vulnerability explanation of VDExplainer. The equation for the navigator to predict the vulnerable probability is shown below.

$$\begin{aligned}\pi_\theta(d_i) &= [p(s_i = 0|d_i;\theta), p(s_i = 1|d_i;\theta)]^T \\ &= [\sigma(W * d_i + b), 1 - \sigma(W * d_i + b)]^T\end{aligned} \tag{1}$$

where π is the probability calculation function of the navigator, $\theta = \{W, b\}$ is the learnable parameters, d_i is the decision vector and σ is the activation function. The goal of training is to maximize the following objective function.

$$J(\pi_\theta) = E_{S\sim\pi_\theta}[G(S)] = \int_S P(S|\pi_\theta)G(S) \tag{2}$$

where $P = \{p_1, \ldots, p_i, \ldots, p_L\}$ is the probability sequence provided by the navigator, $G(S)$ is a target gain value that can measure the quality of the explanation provided by the explorer based on the decision sequence S, $E(G)$ is the expectation of the target gain value. Intuitively, the higher $G(S)$ is, the higher the quality of the explanation. Essentially, we want to find a navigator with the parameter θ such that simply updating the parameter θ affects the validity of the navigator π_θ and therefore J. We calculate the derivative of the objective function with respect to the parameter θ as follows.

$$\begin{aligned}\nabla_\theta J(\pi_\theta) &= \nabla_\theta \int_S P(S|\pi_\theta)G(S) \\ &= \int_S \nabla_\theta P(S|\pi_\theta)G(S) \\ &= \int_S P(S|\pi_\theta)\nabla_\theta \log P(S|\pi_\theta)G(S) \\ &= E_{S\sim\pi_\theta}\nabla_\theta \log P(S|\pi_\theta)G(S) \\ &= E_{S\sim\pi_\theta}\sum_{i=1}^{L}\nabla_\theta \log \pi_\theta(s_i|d_i)G(S) \\ &= \frac{1}{N}\sum_{n=1}^{N}\sum_{i=1}^{L} G(S^n)\nabla_\theta \log \pi_\theta(s_i^n|d_i^n)\end{aligned} \tag{3}$$

where S^n is the decision sequence of the nth sample in the dataset and N is the total number of samples. The parameters of the navigator are updated according to the gradient $\nabla_\theta J(\pi_\theta)$ and the learning rate, which determines how fast the parameters move towards the gradient direction in the parameter space.

3.2.3. Target gain value

Currently, existing datasets of vulnerable code from real-world projects include coarse-grained vulnerability labels (i.e. they indicate whether a file, function or code segment contains a vulnerability or not), but do not necessarily provide fine-grained vulnerability labels. For example, the Big-Vul dataset (Fan et al., 2020) indicates the location of vulnerable statements, but the Devign dataset (Zhou et al., 2019) does not. Obviously, it is more accurate to use fine-grained labels to calculate target values to measure the quality of the explanation, but VDExplainer would also like to be able to be used in weakly supervised environments where fine-grained labels are not available. That is why we offer two options for calculating target values, which developers can choose independently, depending on the conditions.

The first can use fine-grained labels to calculate the $G(S)$, and since we need to determine the overall validity of the explanation, we directly use the Line Coverage (LC) as the target value, which is calculated using the following formula.

$$G_1(S) = LC = \frac{|S \cap U|}{|U|} \tag{4}$$

where LC is the line coverage, i.e., the percentage of vulnerable statements in the predicted vulnerability pattern out of all vulnerable statements. U is the sequence of vulnerable statements recorded by the ground-truth label in the training set, so $|S \cap U|$ can be calculated as the number of vulnerable statements contained in the predicted vulnerability pattern. $LC \in [0,1]$, intuitively, the more vulnerable statements the vulnerability pattern contains, the greater the LC.

The second in the setting without fine-grained labels, we calculate the target value using the detection results of the model that need to be explained. This target value calculation is based on the intuition that the evaluation of vulnerability samples by the detection model is based on the vulnerability-related statements, and that masking the vulnerability-related statements will lead to significant changes in the detection results of the detection model. Consequently, VDExplainer takes as its target value the difference between the probability predicted by the detection model for the original sample and the sample after masking out the vulnerability-related statements. The calculation formula is as follows.

$$G_2(S) = DNet(C) - DNet(C - C_S) \tag{5}$$

where C is the complete original sample to be tested, C_S is the vulnerability-related statements predicted by the explorer, $C - C_S$ is the sample after masking the vulnerability-related statements, and $DNet$ denotes a vulnerability detection model.

The gradient can be computed by the above target gain value, thus updating the network parameters of the navigator. Note that in order to

increase the target value (G_1 or G_2), VDExplainer will tend to predict as many statements as possible to be vulnerable during the training process. Therefore, we limit the number of candidate vulnerability-related statements during the exploration process. When this number exceeds a certain threshold, the explorer removes the statement with the lowest vulnerability probability to ensure the quality of the explanation results.

4. Evaluation setup

4.1. Research questions

RQ1: How effective is VDExplainer at explaining Pre-trained model-based vulnerability detection methods?

RQ2: How effective is VDExplainer at explaining DL-based vulnerability detection methods?

RQ3: How does the design of the navigator (including candidate set combination methods) and its accuracy affect the overall performance of the method?

RQ4: How is the time overhead of VDExplainer compared to other methods?

4.2. Vulnerability dataset and vulnerability detection methods that need to be explained

To evaluate the performance of VDEplainer, we perform experiments on two datasets. The first dataset is the widely used Big-Vul dataset at the level of real-world project functions, provided by Fan et al. (2020). The first reason we use this dataset is to enable a fair comparison with Pre-trained model-based approaches such as LineVul (Fu and Tantithamthavorn, 2022). The second reason is that this dataset contains fine-grained labels of vulnerabilities at the statement level, which allows the evaluation of our explanation methods. Other datasets (e.g. Devign (Zhou et al., 2019) and Reveal (Chakraborty et al., 2021)) provide only function-level labels, which are not suitable for the scope of our study (although our method can be applied to these datasets, the lack of fine-grained labels does not allow us to evaluate the results of the explanation provided). The dataset is collected from 348 open-source GitHub projects containing a total of 188,636 C/C++ functions.

The second dataset is a slice-level semi-synthetic dataset we built ourselves, which includes synthetic codes and real-world project codes, mainly collected from the National Vulnerability Database (NVD) (NVD, 2025) and Common Vulnerabilities and Exposures (CVE) (CVE, 2025). Following the use of program slicing techniques in Refs. Li et al. (2021c) and Zheng et al. (2021a), we generate sliced code segments for the programs in the dataset using six types of vulnerability candidate syntax characteristics (including API/Library Function Call (FC), Array Usage (AU), Pointer Usage (PU), Arithmetic Expression (AE), Function Parameter (FP) and Function Return (FR)) as slicing criterions to improve the performance of the coarse-grained vulnerability detection model. All sliced code segments that contain more than one vulnerable statement are labeled as 1, otherwise as 0. On this basis, we generate a total of 403416 sliced code segments. After deduplication and class balancing, we end up with 18927 code segments with vulnerabilities and 46183 code segments without vulnerabilities.

To further verify the method's performance on data that records defect paths, we introduce the D2 A dataset (Zheng et al., 2021b). This dataset is constructed through differential analysis and contains complete defect path records. Due to the inaccessibility of the original link, we use a version preserved by researchers on the Hugging Face platform (https://huggingface.co/datasets/claudios/D2A), whose labels conform to the logic of recording defect paths and are suitable for this study. The D2 A dataset includes 36,719 sample functions in the training set (with 881 vulnerable samples) and 4634 sample functions in the test set (with 117 vulnerable samples).

For models that need to be explained, we choose two Pre-trained model-based models and four DL-based models. To answer RQ1, we choose LineVul (Fu and Tantithamthavorn, 2022) and VulBERTa-CNN (Hanif and Maffeis, 2022) as our vulnerability detection models, respectively. To answer RQ 2, we select three DL-based vulnerability detection methods based on sequence and graph, which are SySeVR (Li et al., 2021c), Devign (Zhou et al., 2019) and IVDetect (Li et al., 2021a). Due to space limitations, in answering RQ3, we only show the experimental results of VDExplainer on real-world project datasets. To answer RQ4, we evaluate the time costs of various baseline methods and VDExplainer on both the real-world project dataset and the semi-synthetic dataset.

4.3. Baseline method for explanation

To evaluate the performance of the proposed framework, we select eight explanation methods that can explain deep learning model for comparisons. The first three are model-dependent explanation methods, i.e. Attention (Fu and Tantithamthavorn, 2022) (ATT), Layer Integrated Gradient (Sundararajan et al., 2017) (LIG), and Saliency (Simonyan et al., 2013). Among them, ATT is the explanation method provided by LineVul, which uses the self-attention mechanism within the Transformer architecture to find vulnerability-related statements in the code. LIG and Saliency are both classical explanation methods.

The remaining five are model-independent methods, three of which are based on SHAP values to assess feature importance, and two of which specialize in the explanation of GNNs. Among them, the difference between kernelSHAP (Lundberg and Lee, 2017) (K-SHAP), DeepliftSHAP (Lundberg and Lee, 2017) (D-SHAP), and GradientSHAP (Lundberg and Lee, 2017) (G-SHAP) is that different methods are used to approximate the SHAP value. GNNexplainer (Ying et al., 2019) and PGExplainer (Luo et al., 2020) both explain GNN by extracting important edges, GNNexplainer focuses on the explanation of individual samples, while PGExplainer focuses more on explanation using a parameterized explainer.

4.4. Evaluation metrics

The evaluation metrics for the explanation method are intended to reflect whether the vulnerability pattern provides richer and more accurate explanation information that is conducive to vulnerability analysis and diagnosis. Therefore, based on the manifestation of different aspects of effectiveness, we use four evaluation metrics to measure VDExplainer.

Fidelity (FD) (Li et al., 2022): Intuitively, the vulnerability pattern is the key part of the code that has a significant impact on the results of vulnerability detection. Therefore, if the vulnerability pattern is masked, the detection results of the model are likely to change significantly, and fidelity is a measure of this phenomenon. The higher the fidelity, the more important the extracted vulnerability pattern is. Let N be the number of samples. The fidelity formula is as follows.

$$Fidelity = \frac{1}{N}\sum_{i=1}^{N} |DNet(C) - DNet(C - C_S)| \tag{6}$$

K-Accuracy(A(k)): K-Accuracy measures the percentage of samples where at least one actual vulnerable statement appears in the vulnerability pattern containing k statements. 5-ACC (A(5)) and 10-ACC (A(10)) indicate the accuracy of the extracted vulnerability pattern when the vulnerability pattern contains 5 and 10 statements respectively.

Line Coverage(LC): This metric measures the ratio of the number of vulnerable statements in the predicted vulnerability pattern to the number of actual vulnerable statements. To apply the LC metric to all explanation methods, we further derive k-LC (LC(k)), which indicates k statements are selected as the extracted vulnerability patterns. Still using Fig. 1 as an example, assume that the predicted vulnerability

Table 1
Detection results of pre-trained model-based vulnerability detection models that need to be explained (metrics unit:%).

Dataset	Model	A	P	R	F1
Big-Vul	LineVul	99.09	97.12	86.35	91.42
	VulBERTa	99.04	95.81	86.64	90.99
Semi-Syn	LineVul	97.62	88.07	97.50	92.55
	VulBERTa	97.58	88.24	96.98	92.40

pattern contains k statements, of which the LC(k) metric is equal to 1 as long as it contains the 34th statement of the code. This is because LC allows a certain number of contexts to be present in the vulnerability pattern to facilitate the explanation of how the vulnerability occurred, and it is not required that the predicted vulnerability pattern contains only the vulnerability statements. The k values are also taken to be 5 and 10.

$$LC(k) = \frac{|S(k) \cap U|}{|U|} \tag{7}$$

Time: This metric will measure the time it takes for the method to extract the vulnerability pattern.

4.5. Evaluation configuration

We implement the proposed VDExplainer using the machine with an Intel Core 2.6 GHz CPU and an NVIDIA 3080Ti GPU. Taking into account the computational efficiency of VDExplainer and the results of parameter adjustment during the experiments, we make the following parameter settings. We train the model using the Adam optimizer with a learning rate of 0.000001. The decision vector is generally twice as large as the statement vector. For example, for the LineVul model, the dimension of the statement vector is 768 and the dimension of the decision vector is 1536. for the VulBERTa-CNN model, the dimension of the statement vector is 300 and the dimension of the decision vector is 600. Other DL-based methods also follow this rule.

5. Evaluation

5.1. Experiments for answering RQ1

To investigate the answer to RQ1, we use the Pre-trained model-based vulnerability detection methods LineVul and VulBERTa-CNN as the models that need to be explained. We first used Big-Vul, an open-source real-project dataset released by LineVul, which was divided into training, validation and testing sets by LineVul at the time of release. In order to make a fair comparison with LineVul, we directly used the training, validation and test sets divided by LineVul, and downloaded the trained models published by LineVul for experiments. For the semi-synthetic dataset, we divided it in the ratio of 80%:10%:10%, and their coarse-grained detection results are shown in Table 1.

As shown in Table 1, LineVul and VulBERTa-CNN achieved good detection results on the semi-synthetic and Big-Vul datasets. The above A, P, R, and F1 are commonly used metrics in coarse-grained detection tasks, denoting accuracy, precision, recall, and F1 values, respectively. Since the model and dataset partitioning published in LineVul were used, we obtained exactly the same experimental results as in the original LineVul paper (F1 score of 91% and recall of 86%). We then used VDExplainer and other six explanation methods to explain the detection results. Note that PGExplainer and GNNexplainer cannot be applied to LineVul and VulBERTa-CNN as they are methods used to explain GNN model.

Tables 2 and 3 summarize the effects of using different explanation methods for LineVul and VulBERTa-CNN on Big-Vul and semi-synthetic datasets. Where ATT denotes attention, LIG denotes layer integrated gradient, K-SHAP denotes kernel SHAP, D-SHAP denotes deeplift SHAP, G-SHAP denotes gradientSHAP, VDE(G2) denotes the VDExplainer trained using the detection results, VDE(G1) denotes the VDExplainer trained using LC. The experimental results show that VDExplainer has a significant performance advantage over other explanation methods. For the LineVul model, on the real-world project dataset, the 5-accuracy and 10-accuracy of VDExplainer reach 75% and 78% (81% and 84% for VulBERTa-CNN), outperforming the other baseline methods by 25%–41% and 8%–26% (36%–55% and 19%–37%

Table 2
Results of different explanation methods for two Pre-trained model-based models that need to be explained on Big-Vul dataset.

Model	Method	FD	A(5)	A(10)	LC(5)	LC(10)
LineVul	ATT	0.122	46%	65%	0.484	0.661
	LIG	0.107	36%	53%	0.332	0.499
	Saliency	0.129	36%	58%	0.344	0.553
	K-SHAP	0.114	50%	70%	0.504	**0.687**
	D-SHAP	0.096	35%	57%	0.345	0.562
	G-SHAP	0.103	34%	52%	0.343	0.522
	VDE(G2)	**0.135**	54%	71%	0.331	0.507
	VDE(G1)	0.051	**75%**	**78%**	**0.510**	0.594
VulBERTa	ATT	**0.117**	45%	65%	0.476	0.658
	LIG	0.080	33%	53%	0.337	0.535
	Saliency	0.114	32%	56%	0.316	0.547
	K-SHAP	0.085	36%	58%	0.354	0.564
	D-SHAP	0.085	26%	47%	0.258	0.464
	G-SHAP	0.087	30%	47%	0.289	0.462
	VDE(G2)	0.063	59%	74%	0.369	0.552
	VDE(G1)	0.027	**81%**	**84%**	**0.587**	**0.685**

Table 3
Results of different explanation methods for two Pre-trained model-based models that need to be explained on semi-synthetic dataset.

Model	Method	FD	A(5)	A(10)	LC(5)	LC(10)
LineVul	ATT	0.195	48%	78%	0.420	0.708
	LIG	0.234	52%	77%	0.539	0.742
	Saliency	0.322	61%	84%	0.566	0.765
	K-SHAP	0.149	42%	70%	0.375	0.630
	D-SHAP	0.213	52%	68%	0.463	0.616
	G-SHAP	0.287	54%	71%	0.526	0.694
	VDE(G2)	**0.331**	**80%**	86%	**0.703**	0.797
	VDE(G1)	0.329	77%	**90%**	0.681	**0.844**
VulBERTa	ATT	0.272	48%	79%	0.424	0.715
	LIG	0.149	44%	78%	0.409	0.711
	Saliency	0.271	64%	82%	0.564	0.739
	K-SHAP	0.266	45%	78%	0.405	0.706
	D-SHAP	0.239	57%	72%	0.516	0.664
	G-SHAP	0.234	56%	74%	0.520	0.703
	VDE(G2)	0.374	78%	85%	0.680	0.810
	VDE(G1)	**0.445**	**80%**	**87%**	**0.696**	**0.826**

Table 4
Results of different explanation methods for two Pre-trained model-based models that need to be explained on D2A dataset.

Model	Method	FD	A(5)	A(10)	LC(5)	LC(10)
LineVul	ATT	0.228	37%	63%	0.318	0.399
	LIG	0.189	33%	57%	0.266	0.359
	Saliency	0.148	32%	60%	0.244	0.396
	K-SHAP	0.216	36%	59%	0.307	0.441
	D-SHAP	0.238	43%	71%	0.339	0.425
	G-SHAP	0.169	36%	57%	0.306	0.428
	VDE(G2)	**0.239**	58%	78%	0.329	0.486
	VDE(G1)	0.194	**69%**	**85%**	**0.374**	**0.517**
3	ATT	0.146	38%	64%	0.225	0.291
	LIG	0.157	47%	63%	0.193	0.320
	Saliency	0.149	40%	45%	0.207	0.249
VulBERTa	K-SHAP	0.136	35%	62%	0.222	0.328
	D-SHAP	0.133	39%	47%	0.167	0.251
	G-SHAP	0.140	35%	56%	0.180	0.262
	VDE(G2)	**0.362**	50%	68%	0.269	0.351
	VDE(G1)	0.143	**68%**	**79%**	**0.343**	**0.451**

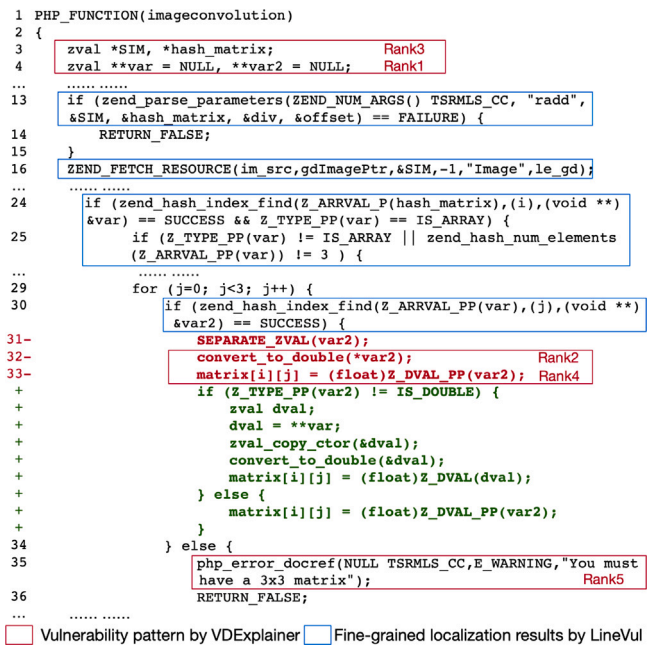

Fig. 3. Vulnerable code for CVE-2014–2020. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

for VulBERTa-CNN, respectively). On the semi-synthetic dataset, the 5-accuracy and 10-accuracy of VDExplainer reach 77% and 90% (80% and 87% for VulBERTa-CNN), outperforming the other baseline methods by 16%–35% and 6%–22% (16%–36% and 5%–15% for the VulBERTa-CNN). Supplementary experiments on the D2 A dataset in Table 4 show that VDExplainer still significantly outperforms baseline methods on the LineVul and VulBERTa models. Taking LineVul as an example, the 5-accuracy (A(5)) and 10-accuracy (A(10)) of VDExplainer reach 69% and 85% respectively, which are higher than those of methods like ATT (37%/63%); the line coverage (LC(5), LC(10)) also outperforms the baselines. These results indicate that VDExplainer can still effectively extract vulnerability-related statements in scenarios with recorded defect paths, verifying the method's wide applicability.

Fig. 3 shows an example of vulnerable code for CVE-2014-2020. The vulnerable statements are in red font and the patched statements are in green font. The vulnerability is described as "This code does not check data types, which might allow remote attackers to obtain sensitive information by using a (1) string or (2) array data type in place of a numeric data type". Consequently, the patched statements check the type of data pointed to by *var2* via the function *Z_TYPE_PP*, and direct use of *var2* without checking leads to the vulnerability. The red boxes in Fig. 3 show the vulnerability pattern predicted by VDExplainer, and the blue boxes show the fine-grained localization results given by LineVul based on attention. As can be seen, VDExplainer can not only find the vulnerable statements during explanation, but also trace the type definition and initialization of *var2*, which helps in understanding and patching the vulnerability. LineVul, on the other hand, did not focus its attention on the vulnerable statements.

In a word, VDExplainer outperforms other model-dependent and model-independent explanation methods on both the real-world project and the semi-synthetic datasets. We believe the main reasons are as follows. First, VDExplainer abstracts the vulnerability pattern extraction task as a sequential decision-making problem, and then combines it with the semantic information extracted by the vulnerability detection model itself to form a general and efficient deep learning explanation method. More specifically, VDExplainer's use of semantic information refers to the statement vectors and vulnerability pattern vectors extracted by borrowing the coarse-grained vulnerability detection model. It is known that Pre-trained model-based models have a deep understanding of semantics, so VDExplainer is effective for explain Pre-trained model-based vulnerability detection methods. Second, VDExplainer uses a learning-based navigator to facilitate the search, which not only speeds up the search but also improves the accuracy of the extracted vulnerability pattern. Specifically, the navigator is able to learn vulnerability patterns through pre-training. During training, the navigator allows the explorer to engage in frequent trial and error to learn which statements can be assembled to form vulnerability patterns and ultimately trigger vulnerabilities. Finally, other explanation methods are not designed for the task of vulnerability detection, which also have an impact on the performance.

In summary, our framework is able to effectively explain Pre-trained model-based vulnerability detection models and achieve greater improvements than other baseline approaches.

Table 5
Detection results of dl-based vulnerability detection models that need to be explained (metrics unit:%).

Model	A	P	R	F1
SySeVR	95.97	81.1	95.73	87.81
Devign	93.38	70.62	96.38	81.51
IVDetect	97.15	87.14	95.23	91.01

5.2. Experiments for answering RQ2

To investigate the answer to RQ2, we use DL-based vulnerability detection methods (including SySeVR (Li et al., 2021c), Devign (Zhou et al., 2019), and IVDetect (Li et al., 2021a)) as the model that to be explained. Considering DL-based methods are difficult to achieve satisfactory results on the real-world project dataset Big-Vul (for example, the maximum F1 value of the DL-based method IVDetect reported in the literature (Fu and Tantithamthavorn, 2022) is only 35), thus we use these DL-based methods only on the semi-synthetic dataset we built to validate our proposed VDExplainer. The coarse-grained detection results are shown in Table 5. The explanation results are shown in Table 6. In addition, none of the three DL-based methods use the attention mechanism, thus they cannot be explained with ATT. PGExplainer and GNNExplainer, which specialize in explaining GNNs, are only used for Devign and IVDetect models.

Experimental results show that VDExplainer significantly outperforms other baseline methods, particularly when explaining GNNs, outperforms PGExplainer and GNNExplainer, i.e. two methods dedicated to GNNs. The main reasons are as follows. First, VDExplainer is supported by a navigator that can learn vulnerability patterns from multiple samples. This helps explorer to make accurate sequence decisions and thus give a more accurate vulnerability explanation. In contrast, GNNexplainer focuses more on explaining isolated samples, making it difficult to understand the vulnerability patterns through generalized features. Second, from a vulnerability analysis point of view, the semantic information contained in the vulnerability-related statements (nodes) plays a greater role in explaining the model than the relationships (edges) between statements. In fact, researchers analyzing the causes of vulnerabilities always tend to know which code statements are related to the vulnerabilities. As a result, unlike PGExplainer and GNNexplainer, which focus more on finding significant edges in graph-based code representations, VDExplainer pays more attention to vulnerability patterns consisting of vulnerability statements (nodes) when explaining detection models.

In summary, the framework explains existing sequence-, tree-, and graph-based vulnerability detection methods and achieves greater improvements over other baseline methods.

Table 6
Results of different explanation methods for three DL-based models that need to be explained on semi-synthetic dataset.

Model	Method	FD	A(5)	A(10)	LC(5)	LC(10)
SySeVR	LIG	0.196	55%	69%	0.460	0.640
	Saliency	0.184	52%	76%	0.388	0.627
	K-SHAP	0.103	47%	71%	0.319	0.549
	D-SHAP	0.256	63%	73%	0.471	0.659
	G-SHAP	0.262	65%	79%	0.528	0.712
	VDE(G2)	0.313	67%	79%	0.565	0.721
	VDE(G1)	**0.396**	**76%**	**83%**	**0.661**	**0.740**
Devign	LIG	0.177	43%	65%	0.320	0.518
	Saliency	0.082	27%	46%	0.212	0.420
	K-SHAP	0.083	44%	70%	0.297	0.537
	D-SHAP	0.222	36%	52%	0.325	0.478
	G-SHAP	0.260	47%	70%	0.378	0.603
	GNNexplainer	0.283	57%	71%	0.483	0.608
	PGExplainer	0.162	42%	67%	0.324	0.543
	VDE(G2)	**0.285**	59%	71%	0.428	0.566
	VDE(G1)	0.201	**63%**	**73%**	**0.563**	**0.623**
IVDetect	LIG	0.279	47%	77%	0.395	0.706
	Saliency	0.078	17%	34%	0.155	0.306
	K-SHAP	0.283	48%	77%	0.396	0.702
	D-SHAP	0.079	17%	34%	0.158	0.309
	G-SHAP	0.290	49%	62%	0.380	0.531
	GNNexplainer	0.364	61%	78%	0.476	0.693
	PGExplainer	0.138	53%	74%	0.360	0.578
	VDE(G2)	**0.392**	64%	80%	0.481	0.715
	VDE(G1)	0.353	**69%**	**84%**	**0.584**	**0.750**

Table 7
Comparative experiments on the exploration mechanism.

Model	Method	FD	A(5)	A(10)	LC(5)	LC(10)
LineVul	Our-method	0.051	75%	78%	0.510	0.594
	Only-statement	0.122	54%	70%	0.351	0.515
	Full-code	0.248	59%	74%	0.367	0.534
VulBERTa	Our-method	0.027	81%	84%	0.587	0.685
	Only-statement	0.070	57%	74%	0.340	0.535
	Full-code	0.015	50%	64%	0.292	0.449

Table 8
Ablation experiments on the navigator.

Model	Method	FD	A(5)	A(10)	LC(5)	LC(10)
LineVul	Our-method	0.051	75%	78%	0.510	0.594
	30%-Noise	0.073	53%	58%	0.288	0.331
	60%-Noise	0.108	45%	53%	0.208	0.301
	Random	0.081	39%	55%	0.226	0.355
	DNet-output	0.083	37%	54%	0.219	0.365
	No-PS	0.127	43%	59%	0.290	0.459
VulBERTa	Our-method	0.027	81%	84%	0.587	0.685
	30%-Noise	0.050	53%	57%	0.289	0.329
	60%-Noise	0.064	48%	55%	0.231	0.320
	Random	0.068	39%	55%	0.226	0.355
	DNet-output	0.071	31%	52%	0.174	0.347
	No-PS	0.061	55%	69%	0.342	0.517

5.3. Experiments for answering RQ3

The navigator is a core component of VDExplainer, responsible for outputting the probability that the current statement is related to vulnerabilities based on the decision vector (a combination of the current statement c_i and the set of candidates V_{i-1}), its accuracy directly affects the overall effectiveness of the method. To verify its performance, we conducted comparative experiments on candidate set combination modes and ablation studies on its accuracy, with results shown in Tables 7 and 8.

To explore whether irrelevant statements in V_{i-1} interfere with the navigator's judgment and verify the rationality of dynamically filtering V_{i-1}, we designed three experimental scenarios. The first, "Only-statement", involves the navigator making predictions based solely on the current statement c_i without combining V_{i-1}, i.e., no contextual information is used. The second, "All-code", sets V_{i-1} as the complete code of the sample (including a large number of vulnerability-irrelevant statements), with the navigator making predictions based on the full context without dynamic filtering. The third, "Our-method", uses V_{i-1} as a dynamically filtered candidate set constructed through the explorer's sequential decision-making, and it is the unmodified VDExplainer method. Experimental results in Table 8 show that on both LineVul and VulBERTa models, Our method outperforms the other two scenarios in all metrics: on LineVul, it achieves 69% in A(5) and 85% in A(10) (vs. 41%/62% for Only-statement and 52%/71% for All-code), with LC(5) = 0.374 and LC(10) = 0.517 (vs. 0.215/0.328 and 0.281/0.403); similar trends are observed on VulBERTa, confirming that dynamic filtering of V_{i-1} reduces noise, while relying on a single statement or full code degrades performance due to insufficient context or excessive noise.

To further verify the importance of the navigator's accuracy, we designed ablation experiments by introducing noise or replacing decision logic. These include "30%-noise"/"60%-noise" (randomly inverting 30%/60% of decisions to simulate 70%/40% accuracy), "Random" (completely random probabilities), "DNet-output" (removing the navigator to use coarse-grained detector probabilities), and "NoPS" (disabling probability sampling during training). As shown in Table 9, the original navigator (Our) outperforms all ablation groups: on LineVul, its A(10) = 85% is much higher than 30%-noise (72%), 60%-noise (58%), Random (41%), DNet-output (53%), and NoPS (61%); on VulBERTa, its A(10) = 79% outperforms 30%-noise (67%), 60%-noise (51%), Random (38%), DNet-output (49%), and NoPS (56%). These results confirm that the navigator's accuracy is critical to the method's effectiveness—any reduction in accuracy or replacement with alternative strategies leads to significant performance degradation, highlighting the irreplaceable role of the original navigator in ensuring VDExplainer's effectiveness.

In summary, both the navigator's design with dynamically filtered candidate sets and its high accuracy are important, enabling VDExplainer to achieve superior overall performance.

Table 9
Time cost to explain LineVul model on Big-Vul dataset (metrics unit: seconds).

ATT	LIG	Saliency	K-SHAP	D-SHAP	G-SHAP	VDE
273	926	81	16 305	9422	160	852

Table 10
Time cost to explain IVDetect on semi-synthetic dataset (metrics unit: seconds).

LIG	Saliency	K-SHAP	D-SHAP
922	56	1943	1084
G-SHAP	GNNexplainer	PGExplainer	VDE
93	8035	276	689

5.4. Experiments for answering RQ4

To verify the effectiveness of VDExplainer in explaining Pre-trained model-based detection models, we choose LineVul as the model that needs to be explained. To verify the effectiveness of VDExplainer in explaining DL-based detection models, we choose IVDetect as the model that needs to be explained. The reason for choosing IVDetect is to facilitate comparison with the two explanation methods, PGExplainer and GNNexplainer. To make a fair comparison, we count the total execution time over the testing set. The experimental results are shown in Tables 9 and 10.

According to the experimental results, ATT and Saliency have lower time costs in model-dependent methods, and PGExplainer and G-SHAP

require less time in model-independent methods. GNNExplainer, K-SHAP and D-SHAP take longer time. This is because GNNExplainer needs to perform mask detection several times on individual samples to find vulnerability patterns, K-SHAP is a slow, perturbation-based local explanation method, and D-SHAP is also a method that focuses more on local accuracy for individual samples. None of the three methods above can explain multiple samples at the same time, which explains their higher time costs. VDExplainer takes a little longer than some of the methods, again mainly due to the fact that VDExplainer has to go through all the statements during exploration to make decisions, which adds to the time cost, but it is still less than the local explanation methods such as GNNExplainer, K-SHAP and D-SHAP.

In summary, VDExplainer can achieve a trade-off between time cost and performance, i.e. achieving better performance within an acceptable interpretation time.

5.5. Threats to validity and future work

First, although the proposed explanation framework can be extended to other languages, the experiments are mainly conducted on C/C++ dataset. In the future, we will apply VDExplainer to more languages. Second, the decision vector needs to be generated by outputting hidden layer vectors from the code representation network in the vulnerability classification model, thus VDExplainer is more suitable for fine-grained explanation of vulnerability detection results from encoder-only Pre-trained models (e.g.RoBERTa), not suitable for decoder-only Pre-trained models (e.g. GPT). This is the research we need to do in the future. Third, the statement-level labels in the Big-Vul dataset are noisy and imprecise, and some of the vulnerabilities were patched by simply adding statements without removing any. As a result, it is not possible to extract vulnerability lines from these samples. The above issues limit the size of the evaluation set in the experiment. In future work, we will construct a higher quality and larger dataset of real projects ourselves. Finally, for a fair comparison, we faithfully restored the parameters of LineVul from the original paper and reproduced its experimental results. Unfortunately, for VulBERTa-CNN, we did not achieve satisfactory results on Big-Vul using its publicly available code. Therefore, to improve the performance of VulBERTa-CNN, we borrowed the tokenizer used by LineVul to achieve better results.

6. Related work

Existing methods for explaining vulnerability detection models can be broadly categorized into model-dependent and model-independent approaches. Additionally, recent advances in Large Language Models (LLMs) have introduced new paradigms for vulnerability detection and explanation, which we discuss to contextualize VDExplainer within the evolving research landscape.

Model-dependent methods provide explanations by analyzing the internal mechanisms of the vulnerability detection model itself. These approaches are tightly coupled with the model's architecture and parameters, leveraging components like gradients or attention mechanisms to derive insights into decision-making. Techniques such as Layer Integrated Gradients (LIG) (Lundberg and Lee, 2017) and Saliency (Simonyan et al., 2013) quantify the importance of input features by measuring how changes in input affect model outputs. For vulnerability detection, these methods attribute importance scores to code tokens or statements based on gradient backpropagation, but they often struggle to capture semantic dependencies between statements. Models like LineVul (Fu and Tantithamthavorn, 2022) use self-attention mechanisms in Transformers to highlight "important" tokens or statements. The attention scores are interpreted as indicators of relevance to vulnerabilities. However, attention scores are inherently heuristic and may not align with actual vulnerability semantics (Zhou et al., 2025), limiting their reliability for explaining critical code patterns.

Model-independent methods decouple the explanation process from the target detection model, focusing on input–output relationships rather than internal parameters. These methods are generally more flexible but may incur higher computational costs. Approaches like LIME (Ribeiro et al., 2016) and SHAP (Lundberg and Lee, 2017) train interpretable proxy models (e.g., linear models) to approximate the behavior of complex detectors. Variants such as KernelSHAP, DeepLIFT-SHAP, and GradientSHAP (Lundberg and Lee, 2017) have been applied to vulnerability detection, but they often oversimplify the model's decision logic, leading to incomplete explanations. Techniques like GNNExplainer (Ying et al., 2019) and PGExplainer (Luo et al., 2020) identify critical code elements (e.g., edges in program dependence graphs) by iteratively masking features and measuring changes in detection results. While effective for graph-based models (e.g., IVDetect (Li et al., 2021a)), these methods suffer from high computational overhead due to repeated model queries and struggle to generalize across samples (Zhou et al., 2025).

Recent years have witnessed growing interest in leveraging Large Language Models (LLMs) for vulnerability detection and repair, as summarized in Zhou et al.'s comprehensive review (Zhou et al., 2025). LLMs like CodeBERT (Feng et al., 2020), VulBERTa (Hanif and Maffeis, 2022), and GPT-3.5/4 have been adapted to these tasks through three primary strategies.

Fine-tuning, LLMs are fine-tuned on vulnerability datasets to enhance their ability to recognize vulnerable patterns. For example, VulBERTa (Hanif and Maffeis, 2022) is pre-trained on C/C++ code to improve vulnerability detection accuracy. However, explanations derived from fine-tuned LLMs rely on attention scores or hidden states, which lack transparency and fail to explicitly identify critical statements (Zhou et al., 2025). Prompt engineering, zero-shot or few-shot prompting guides LLMs to detect vulnerabilities or generate repairs without parameter updates. For instance, GPT-4 can classify code as vulnerable or non-vulnerable based on natural language prompts (Fu et al., 2023). While convenient, these explanations are often unstructured and dependent on prompt design, making it difficult to isolate specific vulnerability-related statements. Retrieval augmentation, methods like Vul-RAG (Du et al., 2024) integrate external vulnerability knowledge bases to enhance LLM decisions. Explanations here are tied to retrieved knowledge fragments, which may not capture the dynamic dependencies between statements in the target code.

VDExplainer distinguishes itself from existing approaches, including LLM-based methods, through three key innovations. First, unlike LLM fine-tuning or attention-based methods, VDExplainer is compatible with any vulnerability detector (e.g., LineVul, VulBERTa-CNN, or even LLM-based detectors), providing a unified explanation framework. Second, while LLM-based methods focus on function-level detection or unstructured explanations, VDExplainer explicitly identifies vulnerability-related statements and their contextual dependencies via the navigator-explorer mechanism, addressing the "black-box" nature of LLMs (Zhou et al., 2025). Finally, Compared to search-based methods (e.g., GNNExplainer) and retrieval-augmented LLMs, VDExplainer's navigator learns generalized vulnerability patterns from training data, reducing redundant searches and improving explanation efficiency.

By addressing the limitations of model-dependent, model-independent, and LLM-based approaches, VDExplainer advances the state of the art in interpretable vulnerability detection, particularly in scenarios requiring explicit, context-aware, and model-agnostic explanations.

7. Conclusion

Inspired by the manual code review process of security experts, we present VDExplainer, a new framework for explaining vulnerability detection results from Pre-trained model-based and DL-based model. We show how VDExplainer can provide some explanation of vulnerability detection results by transforming an initial classification task into a

sequential decision-making task using specially designed navigator and explorer to identify meaningful statements and provide a fine-grained vulnerability pattern. Numerous experiments show that VDExplainer is able to provide more accurate fine-grained explanations compared to existing state-of-the-art explanation methods.

CRediT authorship contribution statement

Weining Zheng: Writing – review & editing, Writing – original draft, Software, Resources, Project administration, Methodology. **Xiaohong Su:** Writing – review & editing, Resources, Project administration. **Yuan Jiang:** Writing – review & editing, Methodology. **Hongwei Wei:** Writing – review & editing. **Wenxin Tao:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant Nos.62272132)

Data availability

I have shared the link to both our code and data in the paper.

References

Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2021. Deep learning based vulnerability detection: Are we there yet. IEEE Trans. Softw. Eng..

CVE, 2025. Common vulnerabilities and exposures. https://cve.mitre.org/.

Du, X., Zheng, G., Wang, K., Zou, Y., Wang, Y., Deng, W., Feng, J., Liu, M., Chen, B., Peng, X., et al., 2024. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag. arXiv preprint arXiv:2406.11147.

Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. AC/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories. pp. 508–512.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155.

Fu, M., Tantithamthavorn, C., 2022. Linevul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories. pp. 608–620.

Fu, M., Tantithamthavorn, C.K., Nguyen, V., Le, T., 2023. Chatgpt for vulnerability detection, classification, and repair: How far are we? In: 2023 30th Asia-Pacific Software Engineering Conference. APSEC, IEEE, pp. 632–636.

Hanif, H., Maffeis, S., 2022. Vulberta: Simplified source code pre-training for vulnerability detection. In: 2022 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–8.

Li, X., Feng, B., Li, G., Li, T., He, M., 2021a. A vulnerability detection system based on fusion of assembly code and source code. Secur. Commun. Netw. 2021.

Li, P., Yang, Y., Pagnucco, M., Song, Y., 2022. Explainability in graph neural networks: An experimental survey. arXiv preprint arXiv:2203.09258.

Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H., 2021b. Vuldeelocator: A deep learning-based fine-grained vulnerability detector. IEEE Trans. Dependable Secur. Comput..

Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2021c. Sysevr: A framework for using deep learning to detect software vulnerabilities. IEEE Trans. Dependable Secur. Comput..

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. arXiv preprint arXiv:1801.01681.

Lundberg, S.M., Lee, S.-I., 2017. A unified approach to interpreting model predictions. Adv. Neural Inf. Process. Syst. 30.

Luo, D., Cheng, W., Xu, D., Yu, W., Zong, B., Chen, H., Zhang, X., 2020. Parameterized explainer for graph neural network. Adv. Neural Inf. Process. Syst. 33, 19620–19631.

NVD, 2025. National vulnerability database. https://nvd.nist.gov/.

Ribeiro, M.T., Singh, S., Guestrin, C., 2016. " Why should i trust you?" Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1135–1144.

Simonyan, K., Vedaldi, A., Zisserman, A., 2013. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:1312.6034.

Sundararajan, M., Taly, A., Yan, Q., 2017. Axiomatic attribution for deep networks. In: International Conference on Machine Learning. PMLR, pp. 3319–3328.

Yang, S., Cheng, L., Zeng, Y., Lang, Z., Zhu, H., Shi, Z., 2021. Asteria: Deep learning-based AST-encoding for cross-platform binary code similarity detection. In: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, pp. 224–236.

Ying, Z., Bourgeois, D., You, J., Zitnik, M., Leskovec, J., 2019. Gnnexplainer: Generating explanations for graph neural networks. Adv. Neural Inf. Process. Syst. 32.

Zhang, J., Wang, X., Zhang, H., Sun, H., Wang, K., Liu, X., 2019. A novel neural source code representation based on abstract syntax tree. In: 2019 IEEE/ACM 41st International Conference on Software Engineering. ICSE, IEEE, pp. 783–794.

Zheng, W., Jiang, Y., Su, X., 2021a. Vu1SPG: Vulnerability detection based on slice property graph representation learning. In: 2021 IEEE 32nd International Symposium on Software Reliability Engineering. ISSRE, IEEE, pp. 457–467.

Zheng, Y., Mao, J., Liu, Y., Ye, Z., Zhang, M., Ma, S., 2019. Human behavior inspired machine reading comprehension. In: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 425–434.

Zheng, Y., Pujar, S., Lewis, B., Buratti, L., Epstein, E., Yang, B., Laredo, J., Morari, A., Su, Z., 2021b. D2a: A dataset built for ai-based vulnerability detection methods using differential analysis. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, pp. 111–120.

Zhou, X., Cao, S., Sun, X., Lo, D., 2025. Large language model for vulnerability detection and repair: Literature review and the road ahead. ACM Trans. Softw. Eng. Methodol. 34 (5), 1–31.

Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Adv. Neural Inf. Process. Syst. 32.

Zou, D., Zhu, Y., Xu, S., Li, Z., Jin, H., Ye, H., 2021. Interpreting deep learning-based vulnerability detector predictions based on heuristic searching. ACM Trans. Softw. Eng. Methodol. (TOSEM) 30 (2), 1–31.